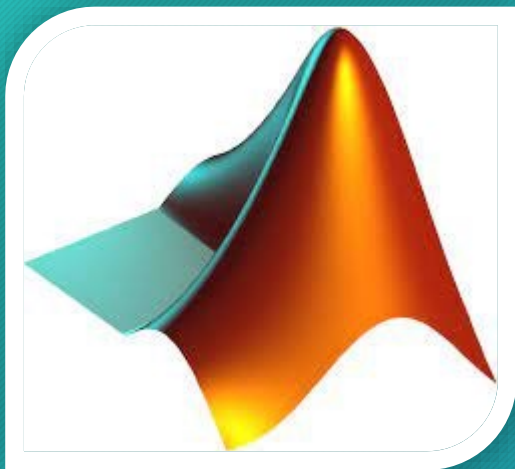




Communication Design

LAB.8

Wireless Communications
OFDM

Ass.Lec.Ammar Alabbassi

Course Outline:

✓ 1- Simulate a Basic Digital Communications Link

- ▶ Implement the essential components of a single-carrier digital communications system, including additive white Gaussian noise.

✓ 2- Pulse-Shaping Filters

- ▶ Incorporate transmit and receive filters into your simulation.

✓ 3- Multipath Channels

- ▶ Model a multipath channel and evaluate the impact on link quality.

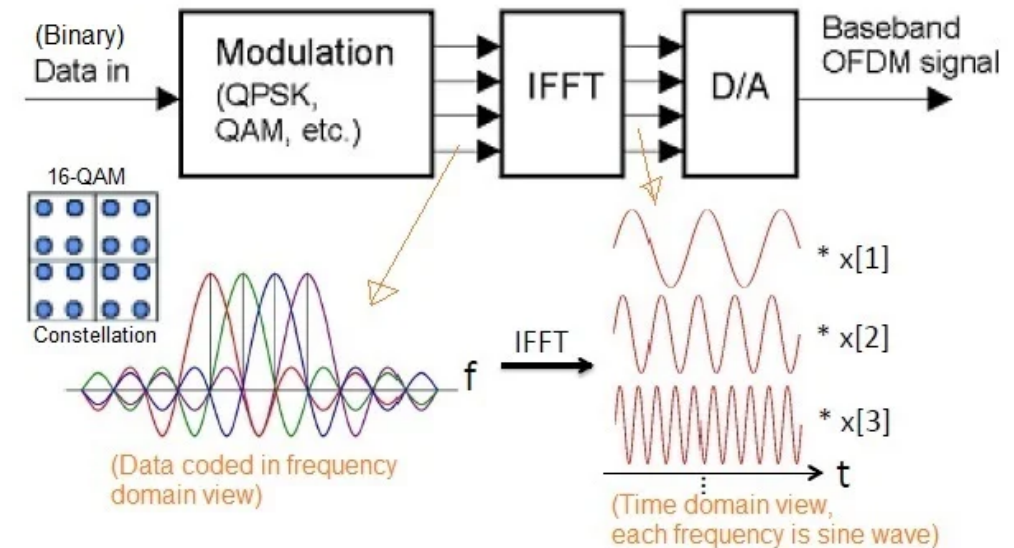
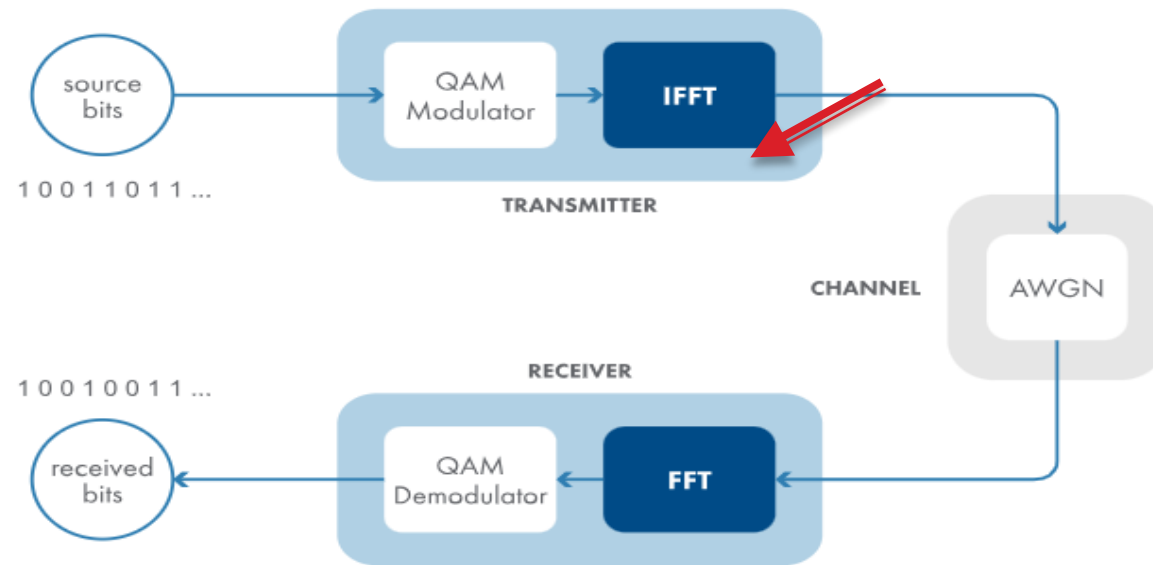
✓ 4- OFDM

- ▶ Implement orthogonal frequency-division multiplexing (OFDM), a multicarrier modulation scheme.

4- OFDM

OFDM Modulation with the Inverse FFT

In this activity, you'll create a simple OFDM system, with an IFFT as the OFDM modulator and an FFT as the OFDM demodulator.



In OFDM, (IFFT) is used at the transmitter to convert the parallel data streams into parallel subcarriers. This process allows for efficient modulation of multiple subcarriers simultaneously. The IFFT operation converts the frequency domain representation of the data into the time domain, which is essential for transmission over the channel. This helps in reducing the complexity of the modulator and makes it easier to implement in hardware.

OFDM Modulation with the Inverse FFT

(Basic link)



Task 1

This code sets up a 16-QAM link.

```
numBits = 32768; % power of 2, to optimize performance of fft/iff  
modOrder = 16; % for 16-QAM
```

```
srcBits = randi([0,1],numBits,1);  
qamModOut = qammod(srcBits,modOrder,"InputType","bit","UnitAveragePower",true);  
scatterplot(qamModOut)  
title("16-QAM Signal")
```

Task 1

Background

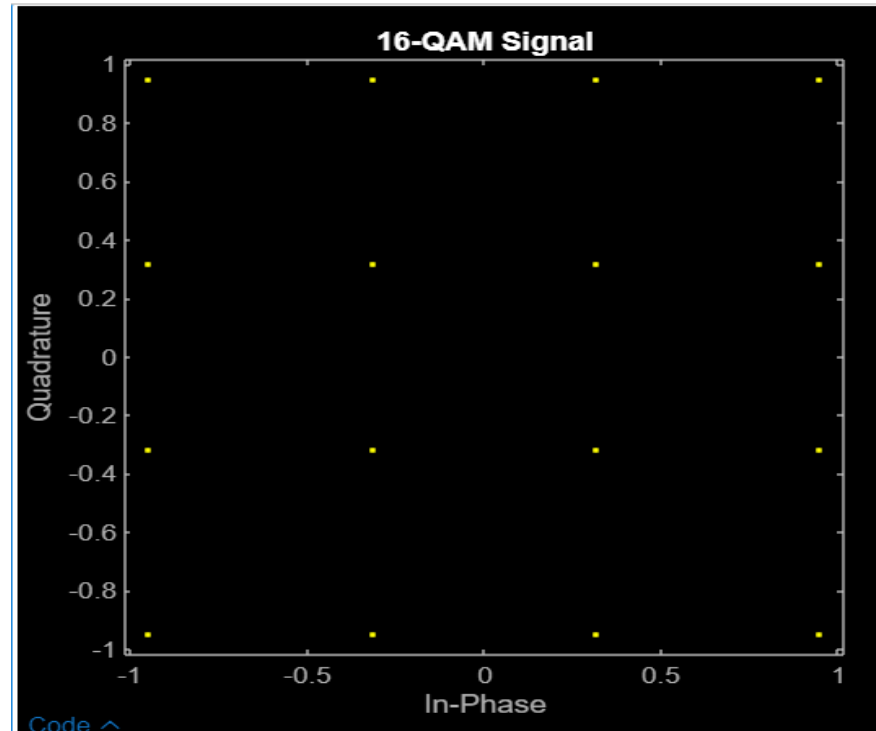
- The `ifft` function computes the inverse discrete Fourier transform of a signal.
`sigOut = ifft(sig)`
- The output is the same size as the input signal.
- Notice that the number of bits in the simulation has changed—it's now a power of 2, which means the length of the QAM signal is also a power of 2. The `ifft` and `fft` functions are significantly faster for signals of certain lengths, including those whose length is a power of 2.

TASK

Compute the IFFT of the 16-QAM signal `qamModOut`. Name the result `ofdmModOut`.

```
ofdmModOut = ifft(qamModOut)
```

Task 1



Apply AWGN.

```
if exist("ofdmModOut","var") % code runs after you complete Task 1
SNR = 15; % dB
chanOut = awgn(ofdmModOut,SNR,"measured");
end
```


Task 2

- Once you've created the variable `ofdmModOut` in the script, it is sent over a noisy channel.
- Next, in the receiver, you need to "undo" the IFFT by performing an FFT.
- The `fft` function computes the discrete Fourier transform, via the fast Fourier transform algorithm.
`sigOut = fft(sig)`

TASK

- Compute the FFT of the received signal `chanOut`. Name the result `ofdmDemodOut`.
`ofdmDemodOut = fft(chanOut)`

Demodulate back into bits, then calculate the BER.

`if exist("ofdmDemodOut","var") % code runs after you complete Task 2`

`qamDemodOut = qamdemod(ofdmDemodOut,modOrder,"OutputType","bit","UnitAveragePower",true);`

`numBitErrors = nnz(srcBits~=qamDemodOut)`

`BER = numBitErrors/numBits`

`end`

numBitErrors = 125

BER = 0.0038

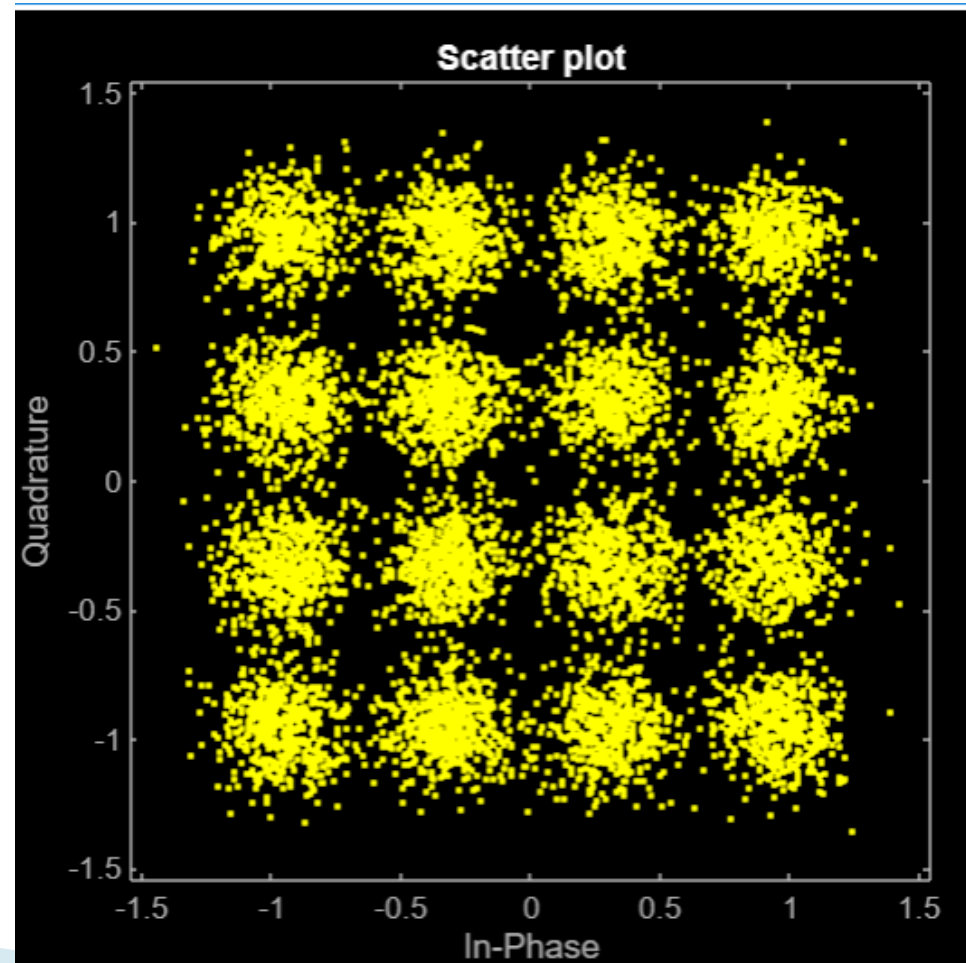
Task 3

- ▶ Since the OFDM demodulator output is a 16-QAM signal, you can look at its scatter plot.

TASK

- ▶ Create a scatter plot of the OFDM demodulator output `ofdmDemodOut`.

`scatterplot(ofdmDemodOut)`



OFDM Modulation with the Inverse FFT

(Channel effects)



Task 1

- Channel: multipath channel and/or AWGN
- The script simulates the same OFDM link from before, but now you can modify the channel model interactively.

TASK

Model a multipath channel by enabling the `hasMultipath` .

OFDM Modulation with the Inverse FFT

%This code sets up a 16-QAM link with OFDM modulation.

numBits = 32768; % power of 2, to optimize performance of fft/ifft

modOrder = 16; % for 16-QAM

srcBits = randi([0,1],numBits,1);

qamModOut = qammod(srcBits,modOrder,"InputType","bit","UnitAveragePower",true);

ofdmModOut = ifft(qamModOut);

Task 1

```
hasMultipath = 0; % disabling multipath channel effects  
hasAWGN = 0;    % disabling AWGN channel effects
```

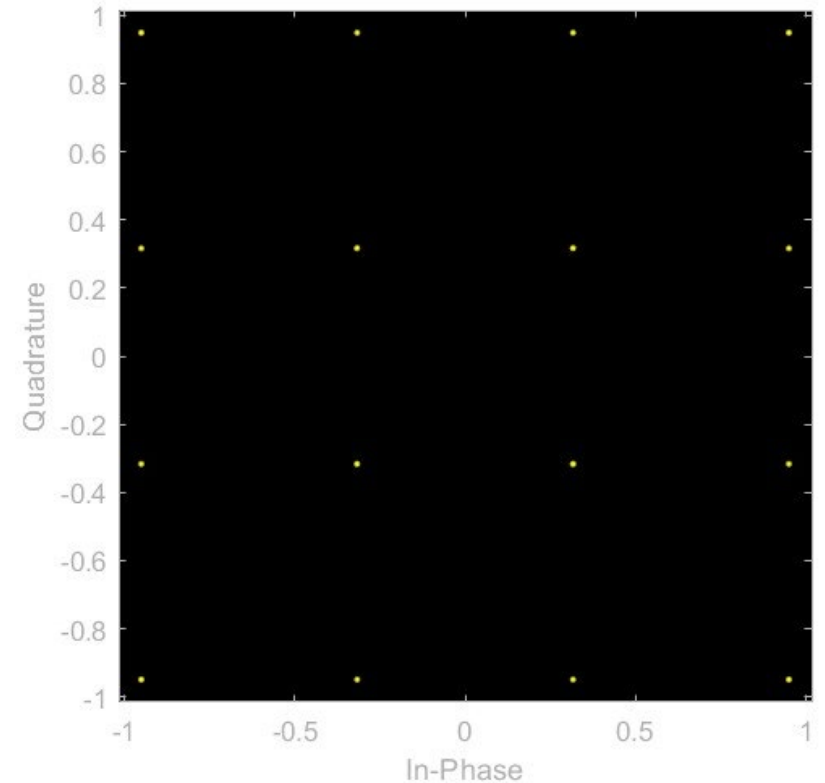


```
SNR = 17;
```

```
chanOut = channel(ofdmModOut,hasMultipath,hasAWGN,SNR);
```

Demodulate the received signal and calculate the BER.

```
ofdmDmodOut = fft(chanOut);  
scatterplot(ofdmDemodOut)  
title("OFDM Demodulator Output")  
qamDemodOut = qamdemod(ofdmDemodOut,modOrder,"OutputType","bit","UnitAveragePower",true);  
numBitErrors = nnz(srcBits~=qamDemodOut)  
BER = numBitErrors/numBits
```



Task 1

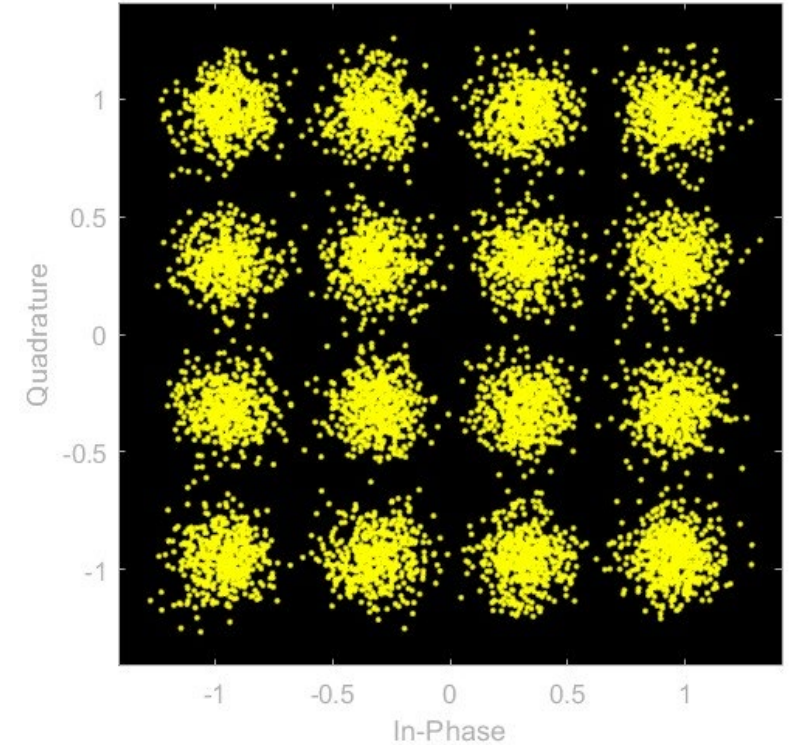
```
hasMultipath = 0; % disabling multipath channel effects  
hasAWGN = 1;    % enabling AWGN channel effects
```

```
SNR = 17;
```

```
chanOut = channel(ofdmModOut,hasMultipath,hasAWGN,SNR);
```

Demodulate the received signal and calculate the BER.

```
ofdmDmodOut = fft(chanOut);  
scatterplot(ofdmDemodOut)  
title("OFDM Demodulator Output")  
qamDemodOut = qamdemod(ofdmDemodOut,modOrder,"OutputType","bit","UnitAveragePower",true);  
numBitErrors = nnz(srcBits~=qamDemodOut)  
BER = numBitErrors/numBits
```



Task 1

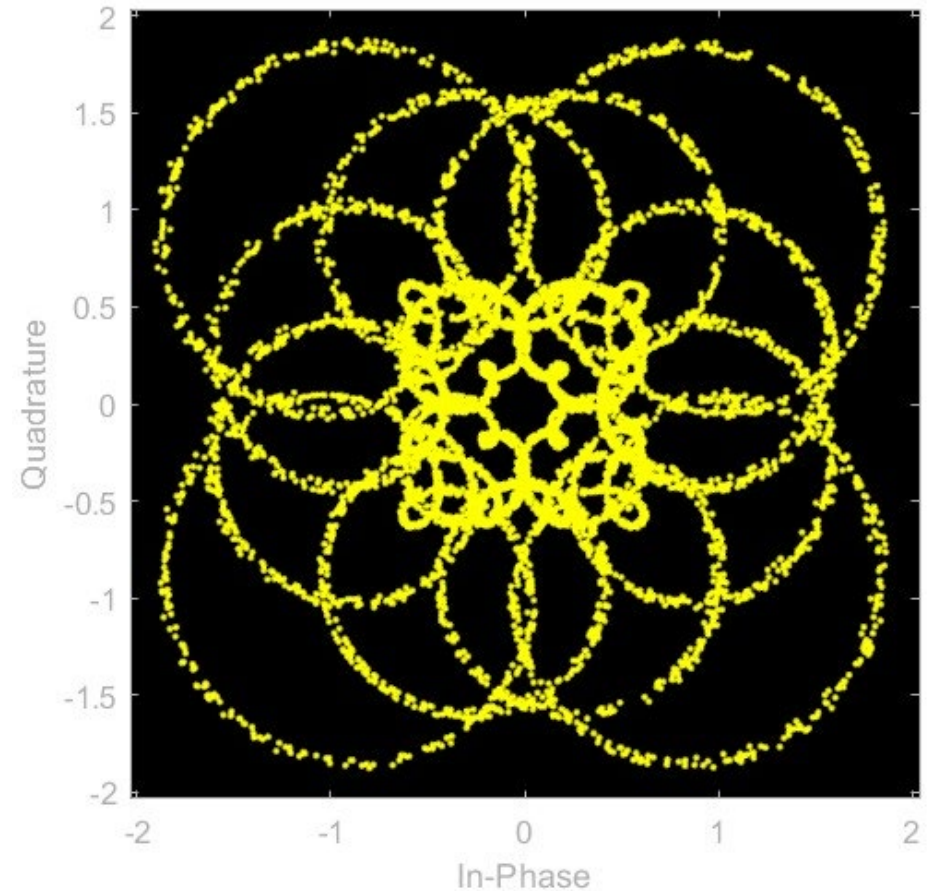
```
hasMultipath = 1; % enabling multipath channel effects  
hasAWGN = 0; % disabling AWGN channel effects
```

```
SNR = 17;
```

```
chanOut = channel(ofdmModOut,hasMultipath,hasAWGN,SNR);
```

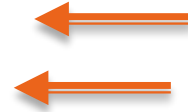
Demodulate the received signal and calculate the BER.

```
ofdmDmodOut = fft(chanOut);  
scatterplot(ofdmDmodOut)  
title("OFDM Demodulator Output")  
qamDemodOut = qamdemod(ofdmDmodOut,modOrder,"OutputType","bit","UnitAveragePower",true);  
numBitErrors = nnz(srcBits~=qamDemodOut)  
BER = numBitErrors/numBits
```



Task 1

```
hasMultipath = 1; % enabling multipath channel effects  
hasAWGN = 1;    % enabling AWGN channel effects
```

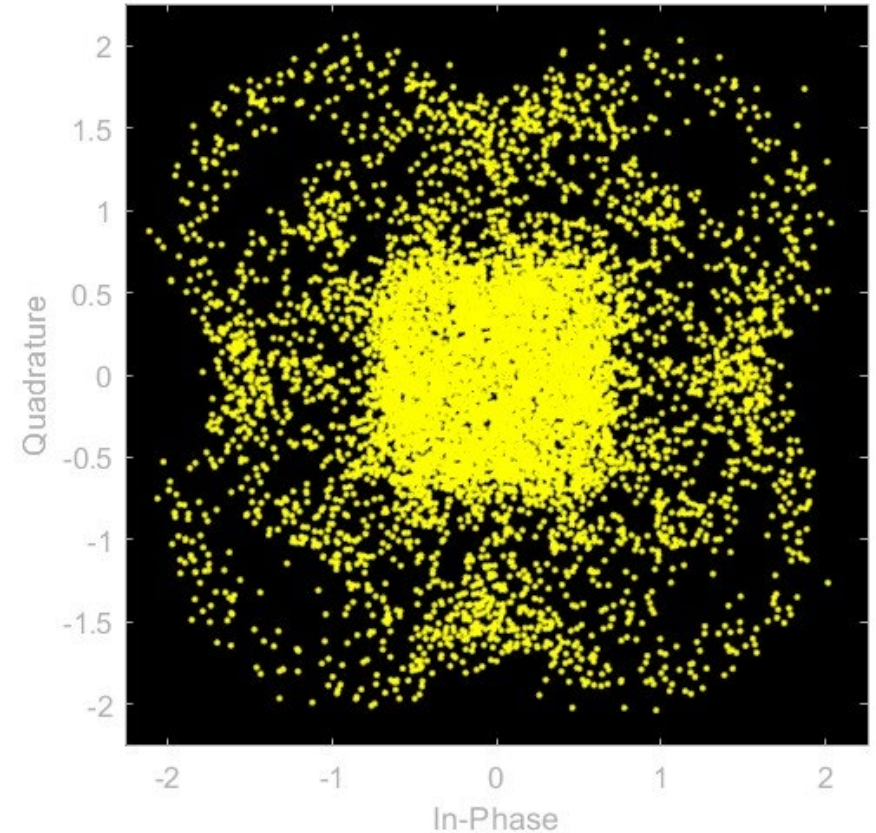


```
SNR = 17;
```

```
chanOut = channel(ofdmModOut,hasMultipath,hasAWGN,SNR);
```

Demodulate the received signal and calculate the BER.

```
ofdmDmodOut = fft(chanOut);  
scatterplot(ofdmDemodOut)  
title("OFDM Demodulator Output")  
qamDemodOut = qamdemod(ofdmDemodOut,modOrder,"OutputType","bit","UnitAveragePower",true);  
numBitErrors = nnz(srcBits~=qamDemodOut)  
BER = numBitErrors/numBits
```



Frequency Domain Equalization



Task 1

Background

- ▶ When designing a practical **OFDM** system, the number of subcarriers (the FFT size) is an important design parameter.
- ▶ The simulation in this activity is designed around the number of **subcarriers**—one **16-QAM** symbol is assigned to each subcarrier, creating a single OFDM symbol. The number of source bits to generate is calculated by multiplying the number of subcarriers by the bits per 16-QAM symbol.

TASK

- ▶ Define the number of subcarriers to be **8192** and store it in the variable **numCarr**. Then calculate the number of source bits to generate and store it in the variable **numBits**.

Frequency Domain Equalization

This code sets the simulation parameters.

```
modOrder = 16; % for 16-QAM
```

```
bitsPerSymbol = log2(modOrder) % modOrder = 2^bitsPerSymbol
```

```
mpChan = [0.8; zeros(7,1); -0.5; zeros(7,1); 0.34]; % multipath channel
```

```
SNR = 15 % dB, signal-to-noise ratio of AWGN
```

```
numCarr = 8192
```

```
numBits = numCarr*bitsPerSymbol
```

Task 2

- ▶ After setting the number of source bits, the following script creates the source bit sequence and modulates it using 16-QAM.

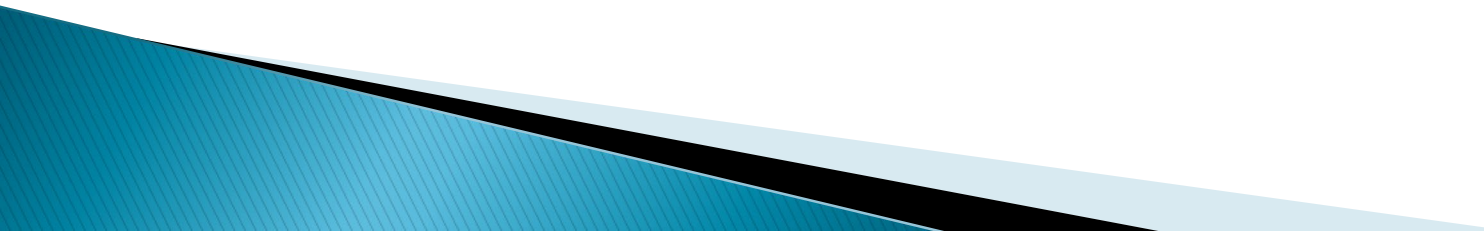
```
srcBits = randi([0,1],numBits,1);  
qamModOut = qammod(srcBits,modOrder,"InputType","bit","UnitAveragePower",true);
```

- ▶ The next step is to perform OFDM modulation, including a cyclic prefix.
- ▶ The `ofdmmod` and `ofdmdemod` functions manage the cyclic prefix for you. All you have to do is specify the length of the cyclic prefix.
- ▶ The cyclic prefix must be at least as long as the impulse response of the channel. But, it should also be reasonably short because it contains duplicate data that will be discarded. In this simulation, the multipath channel impulse response length is 17, so use a value slightly larger than that for the cyclic prefix length.

TASK

Define the cyclic prefix length to be 32 and store it in the variable `cycPrefLen`.

```
cycPrefLen = 32
```



Task 3

- Use the `ofdmmod` function to perform OFDM modulation, specifying the number of subcarriers and the cyclic prefix length.

```
sigOut = ofdmmod(sig,nCarr,cpLen)
```

TASK

- Perform OFDM modulation on the signal `qamModOut`. Specify the number of subcarriers as `numCarr` and the cyclic prefix length as `cycPrefLen`. Name the output signal `ofdmModOut`.

```
ofdmModOut = ofdmmod(qamModOut,numCarr,cycPrefLen)
```

Channel: multipath channel and AWGN

```
if exist("ofdmModOut","var") % code runs after you complete Task 3
mpChanOut = filter(mpChan,1,ofdmModOut);
chanOut = awgn(mpChanOut,SNR,"measured");
end
```

ofdmModOut = 8224x1 complex

	1
1	0.0101 + 0.0024i
2	-0.0009 - 0.0033i
3	-0.0018 + 0.0044i
4	-0.0072 - 0.0076i
5	-0.0029 + 0.0054i
6	0.0028 + 0.0024i
7	0.0109 + 0.0023i
8	0.0064 - 0.0125i

Task 4

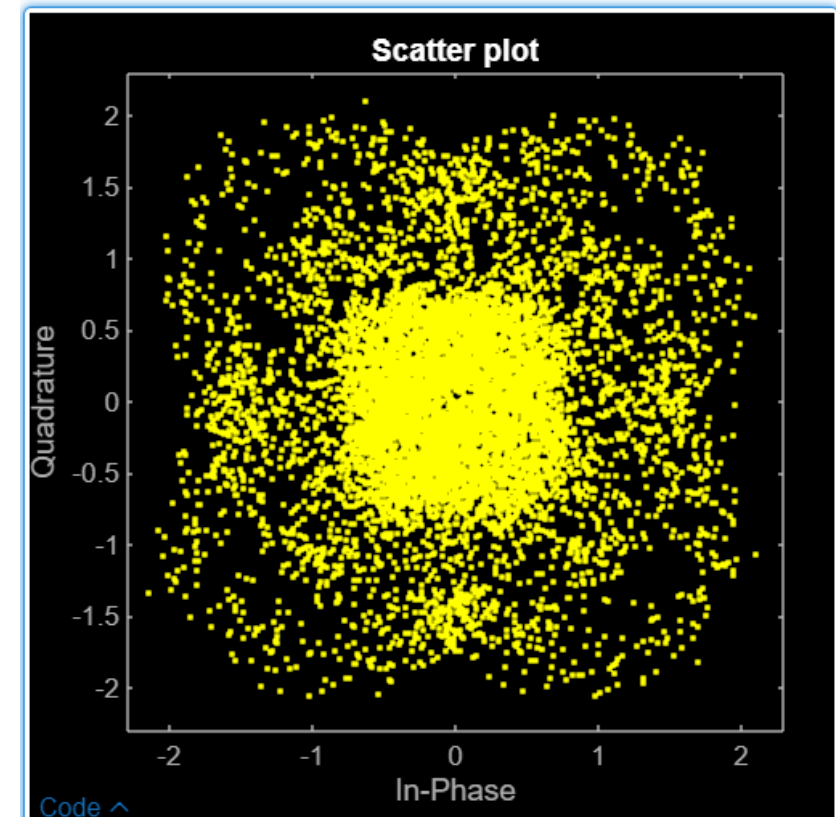
- Next, the **OFDM** modulated signal is sent over the channel to the receiver.
- In the receiver, perform **OFDM demodulation** using the `ofdm demod` function, again specifying the number of subcarriers and the **cyclic prefix length**.

```
sigOut = ofdm demod(sig,nCarr,cpLen)
```

TASK

- Perform OFDM demodulation on the signal `chanOut`. Specify the number of subcarriers as `numCarr` and the cyclic prefix length as `cycPrefLen`. Name the output signal `ofdmDemodOut`.
- Create a **scatter plot** of the OFDM demodulator output.

```
ofdmDemodOut = ofdm demod(chanOut,numCarr,cycPrefLen);  
scatterplot(ofdmDemodOut)
```



Task 5

Equalization

The next step is to equalize using the easy equalization equation.

$$\hat{S} = \frac{R}{\hat{H}}$$

R is the OFDM demodulator output and $\hat{H}$ is the estimated channel frequency response. $\hat{S}$ is the equalized 16-QAM signal.

In this course, you use the known multipath channel impulse response to determine $\hat{H}$. In an actual system, a channel estimation algorithm would use OFDM preamble or pilot signals to derive a channel estimate.

Task 5

Background

- ▶ The equalization happens in the **frequency domain**, so you need to **transform** the channel impulse response using the **fft function**. Set the FFT size by passing it as a second input.

```
fft(mpChan,numCarr)
```

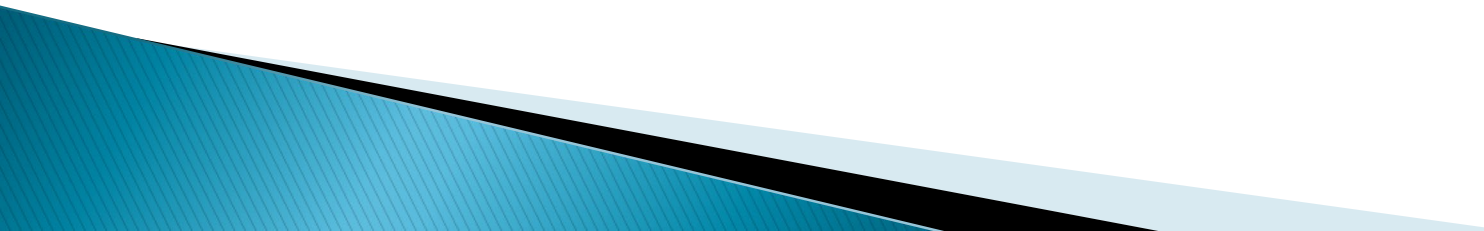
- ▶ The **fft function** output starts with the **zero-frequency component**, but the **ofdm demod** function specifies the zero-frequency component in the **middle of the frequency range**. Use **fftshift** to shift the channel's spectrum so the zero-frequency component is in the center.

```
fftshift(fft(mpChan,numCarr))
```

TASK

- ▶ Transform the multipath channel impulse response **mpChan** to the frequency domain, with the zero-frequency component in the center of the frequency range. Name the result **mpChanFreq**.

```
mpChanFreq = fftshift(fft(mpChan,numCarr));
```



Task 6

- Now you're ready to equalize the signal. To do so, divide it by the channel frequency response, element-wise.
- Divide the two vectors element-wise using the `./` operator.

`x ./ y`

TASK

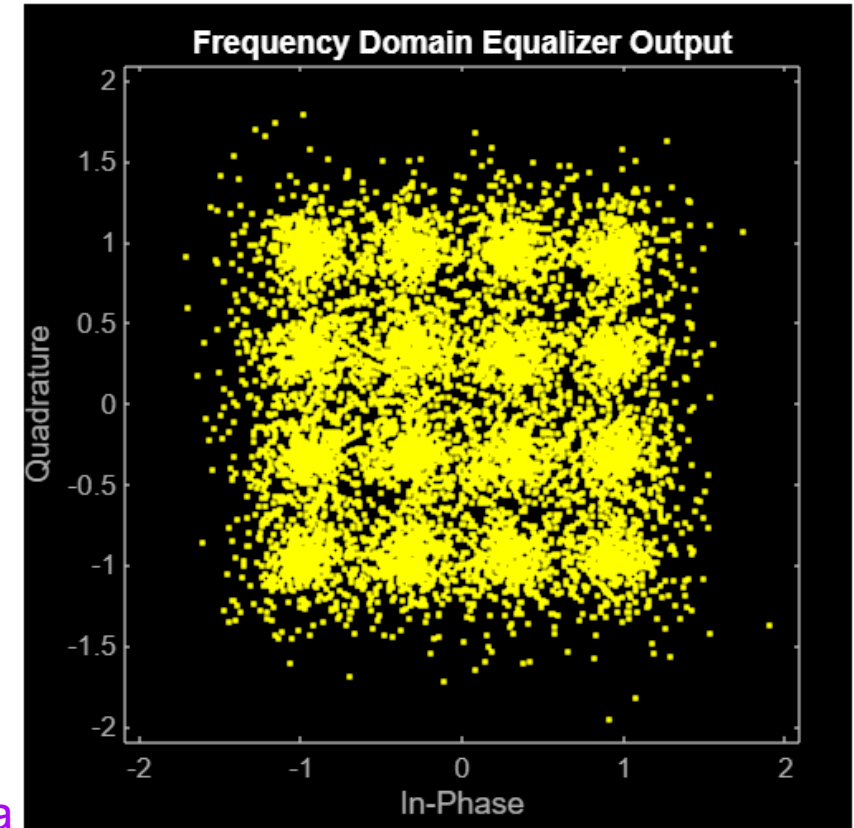
- Equalize the signal `ofdmDemodOut` by dividing by `mpChanFreq`. Name the equalized signal `eqOut`.

- Create a scatter plot of the equalized signal.

```
eqOut = ofdmDemodOut ./ mpChanFreq;  
scatterplot(eqOut)  
title("Frequency Domain Equalizer Output")
```

Demodulate back into bits, then calculate the BER.

```
if exist("eqOut","var") % code runs after you complete Task 6  
qamDemodOut = qamdemod(eqOut,modOrder,"OutputType","bit","UnitAvera  
nnz(srcBits~=qamDemodOut)  
BER = numBitErrors/numBits  
end
```



numBitErrors = 1312
BER = 0.0400

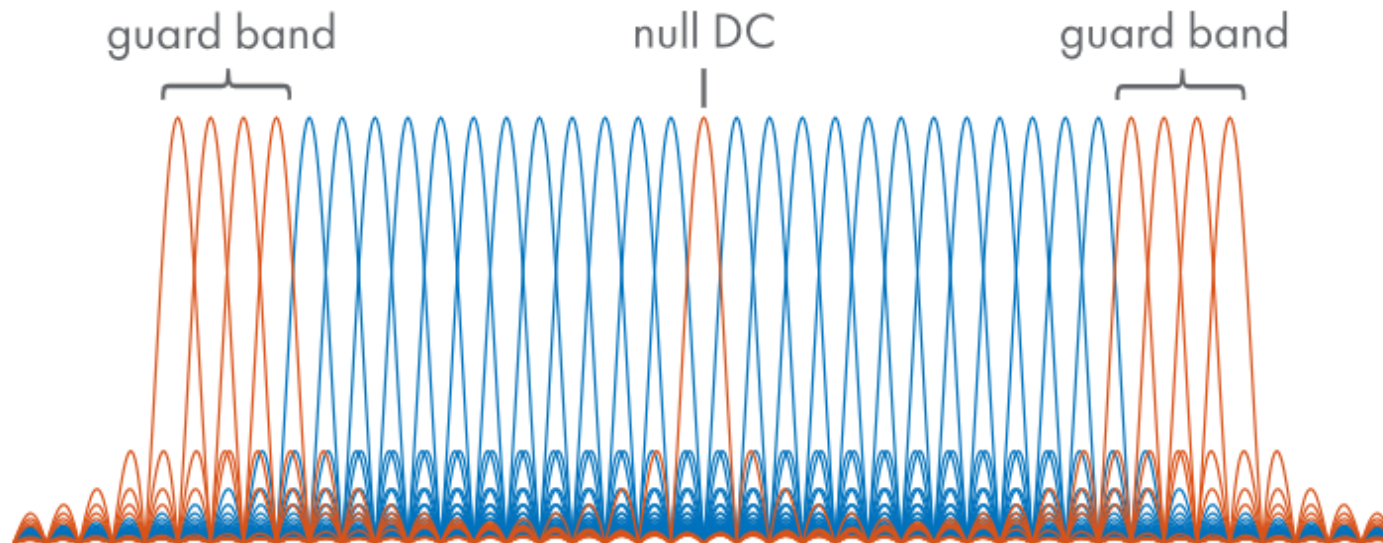
Null Subcarriers



Null Subcarriers

OFDM systems are often designed with additional characteristics beyond the cyclic prefix. For instance, they often have *guard bands*—a set of unused subcarriers at the low and high portions of the allocated bandwidth to minimize interference with adjacent signals. They also often zero out the subcarrier at DC, so that the analog electronics can operate on a signal without a DC component. Practical, standards-based OFDM signals often include pilot signals to aid in channel estimation and equalization.

You can implement all of these design capabilities with the `ofdmmod` function. In this activity, you will consider only guard bands and null DC specification. These subcarriers are called *null subcarriers* because they do not transmit any data.



Task 1

Null Subcarriers

This code sets the simulation parameters.

```
modOrder = 16; % for 16-QAM
```

```
bitsPerSymbol = log2(modOrder) % modOrder = 2^bitsPerSymbol
```

```
mpChan = [0.8; zeros(7,1); -0.5; zeros(7,1); 0.34]; % multipath channel
```

```
SNR = 15 % dB, signal-to-noise ratio of AWGN
```

```
numCarr = 8192; % number of subcarriers
```

```
cycPrefLen = 32; % cyclic prefix length
```

Task 1

Background

- In MATLAB, you identify a **subcarrier** by its **index**. The subcarriers are numbered from left to right, starting with 1 and ending with **numCarr**. The **ofdmmod** and **ofdmdemod** functions order the subcarriers from the most negative frequencies to the most positive frequencies, with the zero-frequency component in the middle.
- To specify the guard band indices, first set the number of subcarriers in each guard band. Then, you can define the guard band indices using the **:** **operator**. The left guard band indices are defined as follows.
1:numGBCarr
- Similarly, you can define the right guard band indices.
(numCarr-numGBCarr+1):numCarr

TASK

- Define the guard band size as **numCarr/16** and store it in the variable **numGBCarr**.
- Define the **left** and **right** guard band indices and store them in the variables **gbLeft** and **gbRight**, respectively.

```
numGBCarr = numCarr/16  
gbLeft = 1:numGBCarr  
gbRight = (numCarr-numGBCarr+1):numCarr
```


Task 2

- The **ofdmmod** function specifies the zero-frequency component in the middle of the frequency range. This means the DC null subcarrier is the center index. When there is an even number of subcarriers, the DC null index is defined as follows.

$$\text{numCarr}/2 + 1$$

- To create the column vector of null subcarrier indices, concatenate the left guard band, DC null, and right guard band indices, and then transpose.

[left dc right]'

TASK

- Define the DC null index and store it in the variable **dcIdx**.
- Create a column vector containing the null subcarrier indices and name it **nullIdx**.

dcIdx = (numCarr/2)+1

nullIdx = [gbLeft dcIdx gbRight]'

Task 3

- The total number of subcarriers is `numCarr`. However, not all subcarriers are data subcarriers.
- This simulation sends one 16-QAM symbol per data subcarrier, which means the length of the source bit sequence is based on the number of data subcarriers.
- To determine the number of data subcarriers, subtract the number of null subcarriers from the total number of subcarriers.
- The number of source bits to generate is then the number of data subcarriers multiplied by the bits per 16-QAM symbol.

TASK

- Calculate the number of data subcarriers and store it in the variable `numDataCarr`. Then calculate the number of source bits to generate and store it in the variable `numBits`.

```
numDataCarr = numCarr - length(nullIdx)  
numBits = numDataCarr*bitsPerSymbol;
```

Task 4

- After setting the number of source bits, the script below creates the source bit sequence and modulates it using 16-QAM.

Create the source bit sequence and modulate using 16-QAM.

`if exist("numBits","var") % code runs after you complete Task 3`

`srcBits = randi([0,1],numBits,1);`

`qamModOut = qammod(srcBits,modOrder,"InputType","bit","UnitAveragePower",true);`

`end`

- Now you can use the `ofdmmod` function to perform OFDM modulation. To specify the null subcarriers, pass the null index vector as a fourth input.
`sigOut = ofdmmod(sig,nCarr,cpLen,nullIdx)`

TASK

- Perform OFDM modulation on the signal `qamModOut`. Specify the number of subcarriers `numCarr`, the cyclic prefix length `cycPrefLen`, and the null subcarrier indices `nullIdx`.
- Name the output signal `ofdmModOut`.

`ofdmModOut = ofdmmod(qamModOut,numCarr,cycPrefLen,nullIdx);`

Task 4

Channel: multipath and AWGN

```
if exist("ofdmModOut","var") % code runs after you complete Task 4
mpChanOut = filter(mpChan,1,ofdmModOut);
chanOut = awgn(mpChanOut,SNR,"measured");
end
```

Task 5

- Next, the OFDM modulated signal is sent over the channel to the receiver.
- Perform **OFDM demodulation** on the received signal using the **ofdm demod** function. Because null subcarriers were used,
- the function needs two additional inputs: **the symbol sampling offset and the null index vector**.
`sigOut = ofdm demod(sig,nCarr, ... cpLen,symOffset,nullIdx)`
- **The symbol sampling offset** is the number of signal samples to discard before performing the FFT. It can be any value from 0 to the cyclic prefix length. To remove information corrupted by the multipath channel, it should be larger than the length of the multipath channel impulse response.

TASK

- Perform OFDM demodulation on the received signal **chanOut**. Specify the number of subcarriers **numCarr**, the cyclic prefix length **cycPrefLen**, and the null subcarrier indices **nullIdx**. For the symbol sampling offset, use the cyclic prefix length **cycPrefLen**.
- Name the output signal **ofdmDemodOut**.

`ofdmDemodOut = ofdm demod(chanOut,numCarr,cycPrefLen,cycPrefLen,nullIdx);`

Task 6

- The next step is to equalize the signal.
- Transform and shift the channel impulse response with the `fft` and `fftshift` functions as you did previously.
`fftshift(fft(mpChan,numCarr))`
- The channel frequency response has `numCarr` frequency components. However, the OFDM demodulator output contains only the data subcarriers. To perform the element-wise division, you need to remove the null subcarrier frequency components from the channel frequency response.
- You can remove specific elements of a vector using indexing.

`x(idx) = []`

TASK

- Transform the multipath channel impulse response `mpChan` to the frequency domain and shift the zero-frequency component to the center of the spectrum. Name the result `mpChanFreq`.
- Remove the elements of `mpChanFreq` that correspond to the null subcarrier indices.

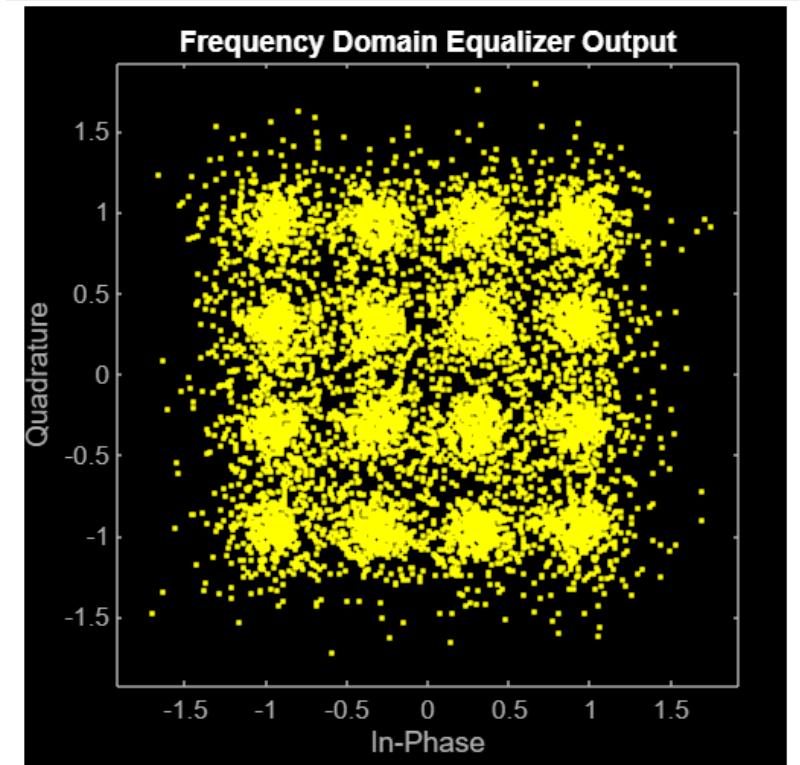
```
mpChanFreq = fftshift(fft(mpChan,numCarr));  
mpChanFreq(nullIdx) = [];
```

Task 7

TASK

- Equalize the signal `ofdmDemodOut` by dividing by `mpChanFreq`. Name the equalized signal `eqOut`.
- Create a scatter plot of the equalized signal.

```
eqOut = ofdmDemodOut ./ mpChanFreq;  
scatterplot(eqOut)  
title("Frequency Domain Equalizer Output")
```



```
numBitErrors = 1009  
BER = 0.0352
```

Demodulate back into bits, then calculate the BER.

`if exist("eqOut","var")` % code runs after you complete Task 7

```
qamDemodOut = qamdemod(eqOut,modOrder,"OutputType","bit","UnitAveragePower",true);
```

```
numBitErrors = nnz(srcBits~=qamDemodOut)
```

```
BER = numBitErrors/numBits
```

```
end
```